Qwen-3.5

> 来源：飞书在线文档；复刻日期：2026 年 10 月 30 日。按要求省略原文代码块。

📌 **模型代码 PR：transformers/pull/43830**

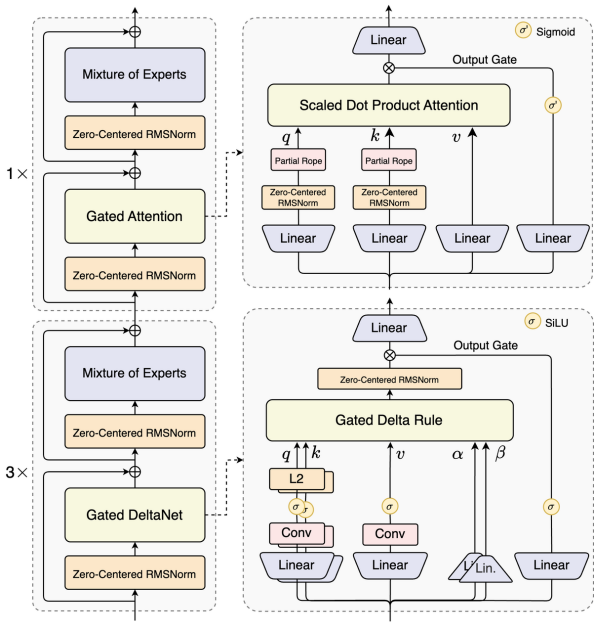
本次开源的代码有 Qwen-3.5 和 Qwen-3.5-MOE 两类多模态大模型，本质上来说：

- **Qwen-3.5 就是多模态版的 Qwen3 Next**，即把 ViT + PatchMerger 接上 Qwen3 Next。自然地，模型也支持 Text-Only 的输出。
- **Qwen-3.5-MOE 在 Qwen-3.5 基础上将 LLM Backbone 的 FFN 替换为 Qwen3 风格的 MoE**（Top-K Router + Multi-Expert + Shared Expert），其中 Shared Expert 是 Qwen3 没有的

1. Qwen-3.5

📌 Qwen-3.5 是一个 Dense 多模态大模型（当然也支持 Text-Only 模式）：

- 多模态架构上相比Qwen3-VL去除了复杂的 `DeepStack` 结构，回归了Qwen2-VL/Qwen2.5-VL的 `ViT + PatchMerger + LLM` 的三段式结构。
- LLM 架构上 Qwen-3.5 参考 Qwen3 Next 使用支持交错 `Gated/Linear Attention` 架构（每 4 层：3 层`GatedDeltaNet` + 1 层`Gated Attention`）
- 整体上看 Qwen-3.5 就是多模态版的 Qwen3 Next。



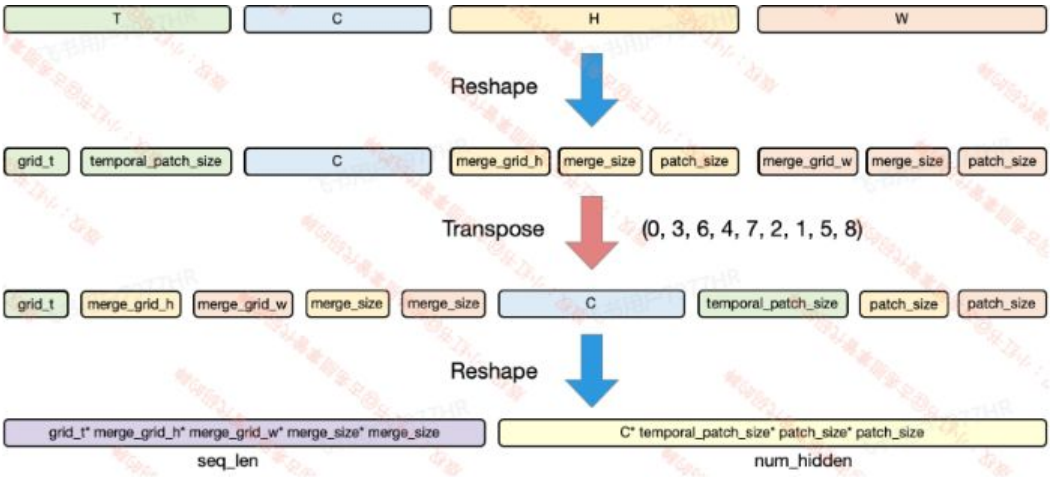
1.1 Vision Encoder

`ViT` 架构回归了Qwen2-VL/Qwen2.5-VL 的 NaViT架构，不再使用Qwen3-VL的`DeepStack`：

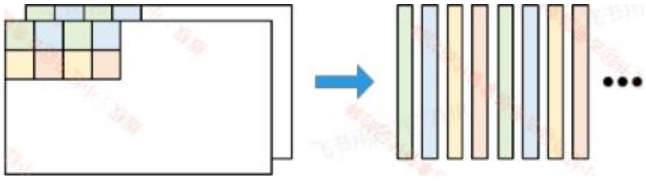
1.2 Projector

Projector 沿用Qwen2-VL/Qwen2.5-VL/Qwen3-VL的 PatchMerger 结构：

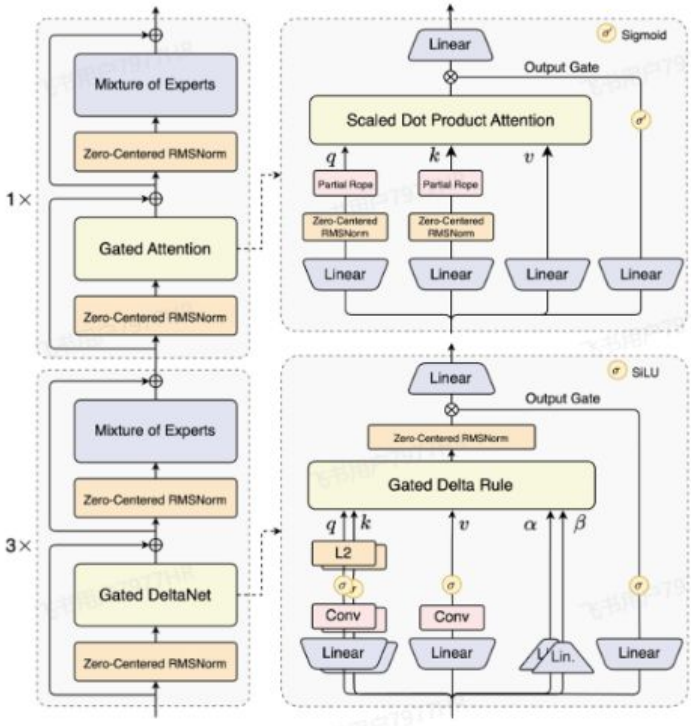
在 ViT 内部（PatchMerger 之前）维度经过切分重组，保证 2x2 的 patch 相邻：具体合并方式是通过 reshape 分割 H，W 维度之后，将两者的 patch size 放在 num_hidden 维度的末尾。而两个 merge_size 放在 seq_len 的末尾，这意味着遍历 reshape 的时候，是 2*2 的 patch 在 seq_len 维度相邻（详细可以参考PatchMerger 是如何合并 2x2 的 Patch 的？）：



即下图的排列方式：



1.3 Qwen-3.5 LLM Backbone

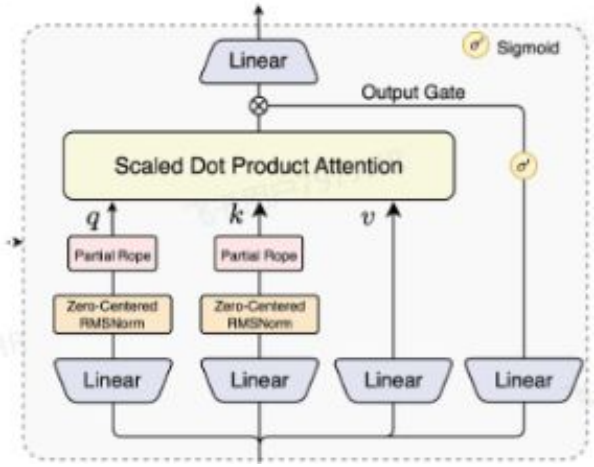


LLM 最重要的特点是交错 `Gated/Linear Attention` 架构 (本质上参考的 Qwen), 默认是每 4 层 Transformer Block 中有 3 层是 `GatedDeltaNet` 有 1 层是 `Gated Attention` (**比例为 3:1**)，参考以下代码：

1.3.1 Gated Attention

首先来看 Gated Attention 的两个关键特点：

- 使用了 QK Norm，并且所使用的 RMSNorm 也是 Qwen3-Next 风格的 **Zero-Centered LayerNorm。**
- `q_proj` 输出维度是 *2，并显式拆出 `gate`，**输出被 learned gate 逐 token/逐 head_dim 调制。**



另外，Zero-Centered LayerNorm 代码实现如下，之所以叫 Zero-Centered，是因为`self.weight = nn.Parameter( torch.zeros(dim))`是进行 0 初始化的，这样一来，初始时就类似恒等映射（scale=1），即标准化，不会因为 RMSNorm 未减均值造成训练不稳定：

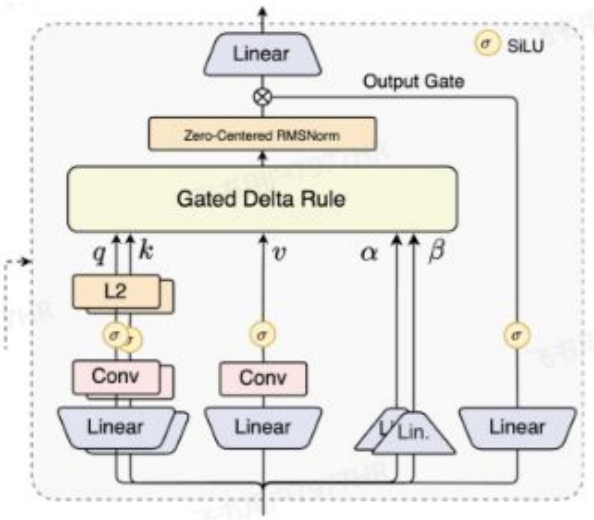
对比标准 RMSNorm 实现：

1.3.2 GatedDeltaNet (Linear Attention)

Gated DeltaNet 用门控（gate）与可学习指数衰减（decay）把历史信息写入一个递推状态，再用查询读出。这样把原本 $O(T^2)$ 的 Softmax Attention 换成 $O(T \cdot d)$ 的递推形式，核心递推（每个 value-head，每个时刻 t）可写成：

$$s_t = \lambda_t \odot s_{t-1} + \beta_t \odot (\Phi(k_t) \otimes v_t),$$
$$y_t = \Phi(q_t) \cdot s_t$$

其中 $0 < \lambda_t < 1$ 是遗忘因子（由 $g_t \le 0$ 决定，$\lambda_t = e^{g_t}$），$\beta_t \in (0,1)$ 是输入门，$\Phi(\cdot)$ 是对 q, k 的特征映射 / 归一化。



核心`gated_delta_rule`代码：

`Qwen3_5GatedDeltaNet` 代码：

2. Qwen-3.5-MoE

关键差异： MoE 版本把 LLM Backbone 的 FFN 从 Dense MLP 换成了 Sparse MoE（Top-K Router + Expert + Shared expert），并且额外提供 router_logits 记录与 load-balancing aux loss 的支持，其余部分一致。

区别在于替换了 FFN：`self.mlp = Qwen3_5MoeSparseMoeBlock(config)`

每个`Qwen3_5MoeSparseMoeBlock` 内部由 3 部分组成：

Router（Top-K）： `Qwen3_5MoeTopKRouter`

- `router_logits = softmax(W_router x)`
- `topk` 选 `num_experts_per_tok` 个专家
- `router_top_value` 归一化作为 token 的 expert 权重

Experts（本质上是多个 FFN 权重）：`Qwen3_5MoeExperts`

- 权重以 3D 张量存储：`(num_experts, ...)`，实际执行把被路由到该 expert 的 token gather 出来跑对应 expert 的 MLP，再 scatter/index_add 回去。

Shared Expert（共享稠密专家，本质上是必须经过的 FFN，但是要做 gating）

- `shared_expert = Qwen3_5MoeMLP(..., shared_expert_intermediate_size)`
- `shared_expert_gate = Linear(hidden, 1)`，用 `sigmoid` 做 gating
- 最终：`expert_output += sigmoid(shared_gate)*shared_expert_output`